; revised .

Predicting Links in Directed Graphs and their Corresponding Directions

Isaac Peterson, Samuel Wise

I. Introduction

Link prediction involves predicting the existence of a relationship in a graph between two nodes. It is an up-and-coming field with roots in social network link prediction [1]. This presented metrics that showed that using relational data could perform better than random when predicting links. Major advancements in link prediction occurred when newer, more effective node embedding methods were developed such as Deepwalk [2] and Node2Vec [3]. These methods were limited however in that they were restricted to features extracted solely from the structure of the graph, unable to utilize the node features themselves. With the advent of increased processing power through graphical processing units (GPU), neural networks began to be able to show promising results, including the GNN [4]. The GNN had the ability to capture both node and structural features to enhance prediction tasks. [5] used GNN to predict hidden links in supply chains with promising results. They show the potential for GNN to accurately predict new links in graphs. Most work in the field of link prediction has focused solely on the prediction of a link, and minimal work has been done on predicting the links direction as well. We present a node labeling method that captures the directionality of the links that leads to significant accuracy increase in direction prediction. The remainder of the paper is organized as follows. Section II covers related work. Section III reiterates the novelty of this research. Section IV goes over the process of link prediction through SEAL. Section V introduces our expansion to SEAL called DirectLabel. Section VI discusses empirical results. Section VII draws the conclusion of this paper.

II. Related Work

Incorporation of directed graphs in predicting existences of links has been seen in several recent works as well. [6] [7] have used directed graphs in order to improve predictive performance in link prediction compared to other benchmark models. These, however, strive to only predict the links themselves, and not their associated direction. Research done in predicting the orientation of a link have also only attempted to do solely that, where they gain results predicting the direction of a known existing link, rather than predicting both the appearance of a link along with its direction, as seen in [8] [9]. The ability to predict direction is important, as most complex networks today are directed. Social networks, one of the more researched complex networks, has seen some work done in directed graphs with some sense of directional prediction [10] [11] [12]. Their use of neighbor-based measures involving triad patterns work well for the concepts of social network link predictions. Note, unlike their methodology, our model will be unaware of which node is the target or the source a priori. For our case of working with world trade and supply chain data, work has been done with other data in various ways in [13] [5]. [13] experiment with link prediction through reconstructing the topological structure of a directed weighted graph with a directed enhanced configuration model. [5] centered their link prediction research around supply chain datasets, where they assume the direction of the links would be known to the practitioner once the link is found and mention directed graphs as a potential future work. We believe that process would be too time-consuming, particularly when working with large datasets, such as our global trade data. We also present a new benchmark dataset from CEPII [14] that can be used for empirical results and further evaluation on future machine learning graph models.

III. Novelty

- We present a new method for predicting the existence and directionality of links in a directed graph.
- We present another benchmark dataset to validate machine learning graph models.

We expand on the graph model SEAL [15] [16], a cutting edge link prediction model, to incorporate new features that allow for more accurate prediction of link direction. We believe the benefits this expansion can provide are added information to help predict the existence of a link and the ability to determine direction as well. This is important in the world of link prediction when considering global trade, as the flow of trades would either not be known, or a field expert would have to observe every predicted link and determine their direction [5]. Allowing for the directionality to be predicted removes the need for unnecessary hours spent and retains the information of flow of trade in a trade directed graph when predicting.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

IV. Link Prediction with SEAL

Given a graph G with vertices $v \in V$ and links $e \in E$. Our task becomes given as input two target nodes, v_x and v_y, predict whether or not a link exists between them. The SEAL model first extracts a subgraph $g \subseteq G, g = \left[V^{'} \in V, E^{'} \in E\right]$, that is constructed from a h-hop neighborhood surrounding v_x and v_y. Given this graph g with associated vertices $v^{'} \in V^{'}$, the goal then becomes creating an information matrix $X^{|V^{'}| \times max(f_l)+d+|\mathbf{x}|}$, where f_l is the node labeling function, d is the embedding dimension, and $\mathbf{x}$ contains the explicit node features.

A. Double Radius Node Labeling (DRNL)

With the subgraph g, we create node labels for $v^{'} \in V^{'}$. Where the label for v_i defined by $f_l(v_i)$ is

$$f_l(v_i^{'}) = 1 + min(d_x, d_y) + \frac{d}{2}\left[\frac{d}{2} + (d \ \% \ 2) - 1\right]$$

where d_x and d_y are the minimum distance between $v_i^{'}$ and our target nodes v_x and v_y respectively, and $d = d_x + d_y$. $\frac{d}{2}$ is the integer quotient and $(d \ \% \ 2)$ is the remainder of d divided by 2. For d_x, $d_y = \infty$, they receive a node label of 0.

We one-hot encode these columns which then become the first $\max f_l(v_i^{'})$ columns in our information matrix X.

B. Random Walk

The next step involves utilizing the expressiveness of random walk embeddings, more specifically node2vec, to create an embedding function $f : v \rightarrow \mathbb{R}^d, \forall v \in V$. The baseline random walk [2] embedding works by walking with uniform probability to a neighboring node of v_i, $v_j \in N(v_i)$, for a total of w iterations, creating walks of length w. Our goal is to find the embedding vectors $z_u \in \mathbb{R}^d$ such that

$$\theta \propto P_r(v|u)$$

Where θ represents the cosine similarity between the embedded vectors z_u and z_v, and $P_r(v|u)$ is the probability that v will exist in the random walk started at node u.

If we set

$$P(v|z_u) = \left(\frac{exp(z_u^T z_v)}{\sum_{n \in V} exp(z_u^T z_n)}\right)$$

where $P(v|z_u)$ is the probability of node v appearing in $N_r(u)$ given the embedding vector z_u, then the goal becomes minimizing the function

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_r(u)} -log\left(\frac{exp(z_u^T z_v)}{\sum_{n \in V} exp(z_u^T z_n)}\right)$$

to get our respective node embeddings.

C. Node2Vec

Node2Vec differs simply by the introduction of two hyper parameters $0 < p, q \leq 1$ that modify the uniform distribution of random walk into a more biased one.

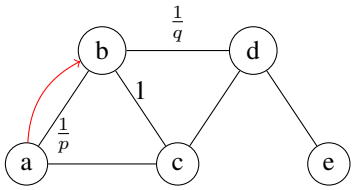


FIGURE 1. Node2Vec Example

Consider the graph above where our walk just traversed the link (a, b), the unnormalized transition probability π_{bx}, where $x \in N(b)$, is defined as

$$\pi_{bx} = \alpha_{pq}(a, x) \cdot w_{bx}$$

$$\alpha_{pq}(a, x) = \begin{cases} \frac{1}{p} & \text{if } d_{ax} = 0 \\ 1 & \text{if } d_{ax} = 1 \\ \frac{1}{q} & \text{if } d_{ax} = 2 \end{cases}$$

d_{ax} being the distance between nodes a and x and w_{bx} the weight on the link (b, x) which is 1 on an unweighted graph. The p and q parameters help balance the search between a breadth first search (BFS) and a depth first search (DFS), which capture local and global graph information. Notice with a smaller p we bias the walk towards a BFS-like walk, and a smaller q introduces bias that leads to a more DFS-like walk. With the Node2Vec embeddings completed the final piece of our information matrix X are the explicit node features that are domain specific to each respective graph resulting in the following matrix,

$$\begin{matrix} \overbrace{\quad\quad}^{\text{NodeLabels}} & & & \overbrace{\quad\quad}^{\text{Node2Vec}} & & & \overbrace{\quad\quad}^{\text{Features}} & & \end{matrix}$$

$$\begin{bmatrix} n_1 & \cdots & n_{max(f_l)} & v_1 & \cdots & v_d & x_1 & \cdots & x_n \\ & \vdots & & & \vdots & & & \vdots & \\ & n_{|V'|} & & & v_{|V'|} & & & x_{|V'|} & \end{bmatrix}$$

D. DeepGCNN

Now that we have our information matrix, we can utilize power of Graph Neural Networks (GNNs) or more specifically Graph Convolution Networks (GCNs) to make link predictions.

A single layer in a GCN is defined as

$$Z = f(\tilde{D}^{-1}\tilde{A}XW)$$

where A is the adjacency matrix of the corresponding subgraph g, $\tilde{A} = A + I$, $\tilde{D}$ is the diagonal degree matrix, f

is our activation function, X is our information matrix, and W is our learned weight matrix. A stacked GCN layer looks like the following

$$Z_{t+1} = f(\tilde{D}^{-1}\tilde{A}Z_tW_{t+1})$$

we end our model with a final fully connected linear layer, outputting a binary classification of 0 or 1 for the existence of a link.

E. Negative Injection

As mentioned in their paper [15], just creating embeddings based on positived links will cause the embeddings to find very quickly which features represent positive links and can lead to overfitting. In order to resolve this issue they introduce the idea of negative injection, where they temporally introduce negative links into the graph over time so the model does not overfit.

V. DirectLabel

A. Methodology

We expand on the results from SEAL, and introduce a new labeling method that we call DirectLabel, that captures the directionality of links and allows for the GNN to learn direction from the graphical structure of the data. It is important to note that in the above process, DRNL is order invariant, in other words, if we define DRNL as

$$f : (v_i, v_x, v_y) \to \mathbb{Z}$$

it can be shown that

$$\forall (v_x, v_y) \in E, f(v_i, v_x, v_y) = f(v_i, v_y, v_x)$$

which means the model will always output the same for these link predictions. In directed graphs however, it is not necessarily true that $(v_x, v_y) = (v_y, v_x)\ \forall e \in E$. As such we need to introduce an *order variant* labeling function such that

$$\forall (v_x, v_y) \in E, f_l(v_i, v_x, v_y) \neq f_l(v_i, v_y, v_x)$$

Our labeling function is defined as

$$f_l(v_i) = \tilde{d}_{ix} - \tilde{d}_{iy}$$

$$\tilde{d}_{ix} = \max \left| \sum_{j=1}^{d_{ix}} \begin{cases} 1 & \text{if } \rightarrow \\ -1 & \text{if } \leftarrow \\ 0 & \text{if } \rightarrow, \leftarrow \end{cases} \right|$$

$$\tilde{d}_{iy} = \max \left| \sum_{j=1}^{d_{iy}} \begin{cases} 1 & \text{if } \rightarrow \\ -1 & \text{if } \leftarrow \\ 0 & \text{if } \rightarrow, \leftarrow \end{cases} \right|$$

Where d_{ix}, d_{iy} are the shortest absolute distance from node v_i and target nodes v_x and v_y respectively. The $\max$ indicating the highest possible absolute value via the shortest path returned from $\tilde{d}_{ix}$. The $\max$ absolute value is for when there are two or more paths that share the shortest absolute distance to node The arrows $\rightarrow, \leftarrow$ indicate that when traveling from node v_i to v_x for example, we add 1 if we are traveling with the direction of the link (v_i, v_x) and subtract one if we are traveling against the direction of the link. In the case when there exists both types of links between the two nodes, as can happen on directed graphs, we do not do anything.

Consider the following example where the link to predict is on the link (a,e).

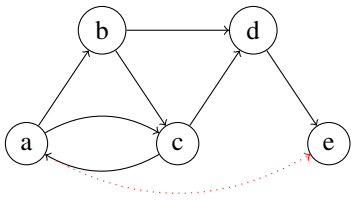


FIGURE 2. DirectLabel Example

$$\tilde{d}_{ax} = 0$$
$$\tilde{d}_{bx} = -1$$
$$\tilde{d}_{cx} = 0$$
$$\tilde{d}_{dx} = (-1) + (-1) = -2$$
$$\tilde{d}_{ex} = (-1) + (-1) + (-1) = -3$$
$$\tilde{d}_{ay} = 1 + 1 + 1 = 3$$
$$\tilde{d}_{by} = 1 + 1 = 2$$
$$\tilde{d}_{cy} = 1 + 1 = 2$$
$$\tilde{d}_{dy} = 1$$
$$\tilde{d}_{ey} = 0$$

$$f_l(a) = 0 - 3 = -3$$
$$f_l(b) = -1 - 2 = -3$$
$$f_l(c) = 0 - 2 = -2$$
$$f_l(d) = -2 - 1 = -3$$
$$f_l(e) = -3 - 0 = -3$$

Notice that if we flip the order, in other words, if our input is the link (e,a) the signs are flipped

$$\tilde{d}_{ax} = 1 + 1 + 1 = 3$$
$$\tilde{d}_{bx} = 1 + 1 = 2$$
$$\tilde{d}_{cx} = 1 + 1 = 2$$
$$\tilde{d}_{dx} = 1$$
$$\tilde{d}_{ex} = 0$$
$$\tilde{d}_{ay} = 0$$
$$\tilde{d}_{by} = -1$$

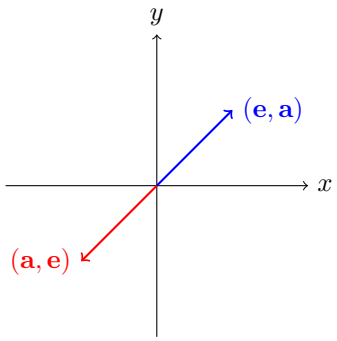
$$\tilde{d}_{cy} = 0$$
$$\tilde{d}_{dy} = (-1) + (-1) = -2$$
$$\tilde{d}_{ey} = (-1) + (-1) + (-1) = -3$$

$$f_l(a) = 3 - 0 = 3$$
$$f_l(b) = 2 - (-1) = 3$$
$$f_l(c) = 2 - 0 = 2$$
$$f_l(d) = 1 - (-2) = 3$$
$$f_l(e) = 0 - (-3) = 3$$

resulting in the following column vectors for the DirectLabel of (a,e) and (e,a) respectively.

$$\begin{bmatrix} -3 \\ -3 \\ -2 \\ -3 \\ -3 \end{bmatrix} \begin{bmatrix} 3 \\ 3 \\ 2 \\ 3 \\ 3 \end{bmatrix}$$

Geometrically speaking DirectLabel forms vectors of opposite sign, maximizing the angle between the two vectors, which is a property we desire to distinguish the two edges from each other.



Because of how SEAL is formulated we can easily inject our DirectLabel into our information matrix X for the following resulting matrix

$$\begin{array}{cccccccccc} \overbrace{}^{\text{DirectLabel}} & & \overbrace{}^{\text{NodeLabels}} & & \overbrace{}^{\text{Node2Vec}} & & & \overbrace{}^{\text{Features}} & & \\ \begin{bmatrix} f_1 \\ \vdots \\ f_{|V'|} \end{bmatrix} & \begin{matrix} n_1 \\ \\ \end{matrix} & \begin{matrix} \dots \\ \vdots \\ n_{|V'|} \end{matrix} & \begin{matrix} n_{max(f_l)} \\ \\ \end{matrix} & \begin{matrix} v_1 \\ \\ v_{|V'|} \end{matrix} & \begin{matrix} \dots \\ \vdots \\ \end{matrix} & \begin{matrix} v_d \\ \\ \end{matrix} & \begin{matrix} x_1 \\ \\ x_{|V'|} \end{matrix} & \begin{matrix} \dots \\ \vdots \\ \end{matrix} & \begin{bmatrix} x_n \\ \\ \end{bmatrix} \end{array}$$

All other aspects of SEAL remain unchanged, except for the last fully connected linear layer of the GCN. The final layer which outputs a binary classification of 0 or 1 for the existence of a link is now interpreted as 0 representing a prediction of no link, and 1 representing a prediction of $\rightarrow$, an outgoing link. Using our example of link (a,e), an output of 1 would mean the model predicted a link directed towards node e with node a as the source node.

B. Problems with DirectLabel

Our goal was to introduce a labeling function $f_l(v_i, v_x, v_y)$ such that

$$\forall (v_x, v_y) \in E, f_l(v_i, v_x, v_y) \neq f_l(v_i, v_y, v_x)$$

however, there is no guarantee that this is the case. For example consider the following subgraph with target link (a,e).

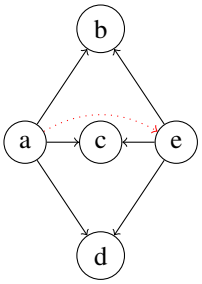


FIGURE 3. DirectLabel Failure

Notice that DirectLabel will return the same labeling for (a,e) and (e,a). This problem of certain isomorphic nodes is a recurring problem in the world of multi-node representations [16], as it was one of the reasons for the implementation of SEAL for the problem over other GNN-based link prediction methods. SEAL's method of node labeling allows for each single-node representation to be aware of the other target node's position in the structure, which makes the representations dependent on one another. It is unique in this case as now the problem relates to retaining information of the directionality of the links, rather than the structure of the subgraphs. In some cases, directionality can be used as a distinguisher between these isomorphic nodes, but still face the same problem when the directions are also symmetric.

VI. Results

REFERENCES

[1] D. Liben-Nowell and J. Kleinberg, "The link-prediction problem for social networks," *Journal of the American Society for Information Science and Technology*, vol. 58, pp. 1019–1031, 2007.
[2] B. Perozzi, R. Al-Rfou, and S. Skiena, "Deepwalk: Online learning of social representations," pp. 701–710, Association for Computing Machinery, 2014.
[3] A. Grover and J. Leskovec, "Node2vec: Scalable feature learning for networks," vol. 13-17-August-2016, pp. 855–864, Association for Computing Machinery, 8 2016.
[4] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Transactions on Neural Networks*, vol. 20, pp. 61–80, 1 2009.
[5] E. E. Kosasih and A. Brintrup, "A machine learning approach for predicting hidden links in supply chain with graph neural networks," *International Journal of Production Research*, 2021.
[6] J. S. Li, J. H. Peng, S. X. Liu, and Z. C. Li, "Predicting missing links in directed complex networks: A linear programming method," *Modern Physics Letters B*, vol. 34, 10 2020.
[7] S. Liu, H. Chen, L. Wu, K. Wang, and X. Li, "Link prediction method fusion with local structural entropy for directed network," *Chinese Journal of Electronics*, vol. 33, pp. 204–216, 1 2024.
[8] X. Wang, X. Zhang, C. Zhao, Z. Xie, S. Zhang, and D. Yi, "Predicting link directions using local directed path," *Physica A: Statistical Mechanics and its Applications*, vol. 419, pp. 260–267, 2015.

[9] F. Guo, Z. Yang, and T. Zhou, "Predicting link directions via a recursive subgraph-based ranking," *Physica A: Statistical Mechanics and its Applications*, vol. 392, pp. 3402–3408, 2013.
[10] D. Schall, "Link prediction in directed social networks," *Social Network Analysis and Mining*, vol. 4, p. 157, 12 2014.
[11] E. Bütün, M. Kaya, and R. Alhajj, "Extension of neighbor-based link prediction methods for directed, weighted and temporal social networks," *Information Sciences*, vol. 463-464, pp. 152–165, 2018.
[12] J. Li, J. Peng, S. Liu, X. Ji, X. Li, and X. Hu, "Link prediction in directed networks utilizing the role of reciprocal links," *IEEE Access*, vol. 8, pp. 28668–28680, 2020.
[13] C. Ding and K. Li, "Estimating multilateral trade behaviors on the world trade web with limited information," *Neurocomputing*, vol. 210, pp. 66–80, 2016.
[14] G. Gaulier and S. Zignago, "Baci: International trade database at the product-level the 1994-2007 version," 2010.
[15] M. Zhang and Y. Chen, "Link prediction based on graph neural networks," 2 2018.
[16] M. Zhang, P. Li, Y. Xia, K. Wang, and L. Jin, "Labeling trick: A theory of using graph neural networks for multi-node representation learning," 10 2020.